PORTFOLIO · HBM DIGITAL DESIGN (RTL DESIGN)

High Speed Link Lite

HBM Base Die Digital Link Slice — Spec · Architecture · RTL · Verification · PD Guide

지원 직무	SK hynix · HBM Digital Design (RTL Design)
지원자	권용현 · 010-6408-8646
저장소	`github.com/yhkwon-git/High_Speed_Link_Lite` · 측정 `commit 14fec6e` (쪽마다 표기)
사용 도구	SystemVerilog · Verilator 5.052 · ModelSim 10.1d · Yosys 0.52 · OpenSTA 3.1 (Nangate45) · Python

※ 숫자마다 측정 commit을 적었다. p.2–20과 p.23–28은 commit 14fec6e의 칩(컨트롤러 v4 · mission PHY · DFT / DFD)이 기준이고, p.21–22는 그 시점의 반영 기록(DBI-AC e30ea96, 리뷰 수정 d1f8e2b)이다. 회귀는 Verilator 5.052와 ModelSim-Altera 10.1d 모두 23 / 23 PASS.

HBM Base Die 디지털 링크 Slice 설계 개요

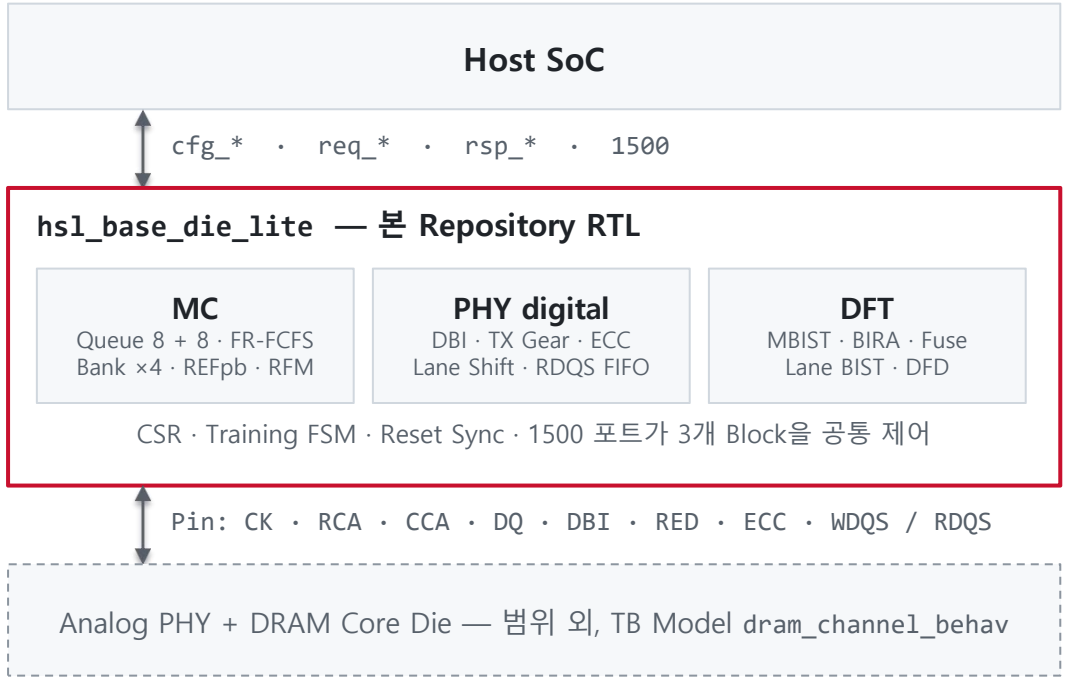
HBM Base Die 1개 Channel을 Study Scale로 축소 — Spec 정의부터 PD Guide까지 단독 수행 가능한 범위로 구성

배경 및 목표

- **Custom HBM 전제** (Memory Controller를 Base Die로 이동, 표준 HBM은 Host SoC 측) — Base Die가 Command Scheduling · Training · Data Encoding · Test를 디지털로 중재
- 고속 PHY는 Analog 비중이 높아, 디지털 담당 영역의 **경계** (Pin 논리값 · Handshake · CDC · Reset · DFT 관측점)을 문서와 RTL로 명확히 정의하는 것이 목표
- 실제 HBM Channel 폭 대신 DQ 8 · BL 4 · Bank 4로 축소, MC | PHY Digital | DFT 구조와 설계 규칙은 동일하게 유지

Out of Scope — docs/01_requirements.md

Multi-channel · 전력(mW) Sign-off · Analog Margin · JEDEC/DFI 호환 선언 — 근거 없는 수치는 기재하지 않음



MC · PHY · DFT

설계 범위 · 목표 3.2 Gb/s/DQ (Controller 800 MHz / PHY 1.6 GHz)

35 모듈 · 4,611줄

RTL 37파일 5,900줄 (주석 제외 4,611줄) · TB 4,237줄 (14fec6e)

23 / 23 PASS

SMOKE · T1–T17 · E7 · E9 · E11 단일 Regression (Verilator 5.052 · ModelSim 10.1d, 14fec6e)

95.5% · 91.4%

Line · Branch Coverage (Toggle 78.8%, commit 14fec6e)

요구사항과 주요 스펙

숫자는 docs/01_requirements.md → hsl_pkg → RTL → TB → SDC 한 방향 · DRAM 타이밍은 컨트롤러의 dram_pkg 하나에서

주요 파라미터 (hsl_pkg)

항목	값
DQ / DBI / RED / ECC	8 / 1 / 1 / 2 → 데이터 핀 10 + ECC 핀 2
CA	행 CA 13 bit {cmd, bank, row} · 열 CA 10 bit {cmd, bank, col, ap}
Strobe	WDQS (쓰기) · RDQS (읽기, CK + tDQSCK)
버스트	BL 4, beat-major (beat 0 = 첫 UI)
Bank · Row · Col	4 (bank group 2 × 2) · 8 bit · 4 bit
호스트 주소	14 bit 워드 주소, XOR 맵 (bank = a[5:4] ^ a[7:6])
데이터 폭	32 bit = DQ_W × BL, 컨트롤러 클럭당 버스트 하나
목표 데이터율	3.2 Gb/s/DQ → raw 3.2 GB/s
클럭	컨트롤러 1.25 ns (800 MHz) · PHY 0.625 ns (1.6 GHz)

클럭 구조 (docs/e12_clocks.md 대조표로 문서 · pkg · TB · SDC 일치 확인)

```
CK              = 3.2 Gb/s ÷ 2    = 1.6 GHz (625 ps)
Controller      = CK ÷ 2          = 800 MHz (1250 ps)
beats_per_burst = BL ÷ DDR_BITS   = 2  → PHY beat 20 bit (2 UI × 10핀)
// RDQS: 위상이 정해지지 않은 세 번째 클럭, 읽기 FIFO로만 넘어감
```

DRAM 타이밍 (컨트롤러 클럭, dram_pkg)

규칙	값	지키는 블록
tRCD · tRP · tRAS · tRC	3 · 3 · 6 · 9	mc_bank (bank마다)
tRTP · tWR	2 · 2	mc_bank
tCCD_S / L · tRRD_S / L	1 / 2 · 2 / 3	mc_chan (bank group별)
tWTR_S / L · tRTW · tFAW	2 / 4 · 4 · 8	mc_chan
tRFC · tRFCpb · tREFI	12 · 8 · 64	mc_refresh (tREFI / 4마다 REFpb)
WL	1	mc_resp (WR 다음 클럭 쓰기 데이터)

기능 요구와 합격 기준

- 트레이닝: RESET → CKE → MRS → ZQCAL → WRLVL → RDLVL → DESKEW → WRTRAIN → MISSION. 요청 포트는 MISSION이면서 phy_en일 때만
- 응답은 요청 순서, 쓰기는 posted. 핀의 명령 간격은 dram_pkg 규칙 이상 (채널 모델이 핀에서 검사)
- 루프백 RD는 마지막 쓰기 버스트를 돌려줌. DQ 레인 하나 고장은 spare(RED)로 수리
- 전력 · 신뢰성: CKE는 RESET에서만 0, 쓰기 beat가 없으면 TX 0, DBI DC · AC, 에러 카운터 16'hFFFF 포화
- 합격: 체크마다 PASS / FAIL, [ALL] PASS 외에는 non-zero exit, 타임아웃과 X는 실패

설계 결정: 클럭 구조, 컨트롤러 재사용, DFI 경계

결정마다 이유와 대가를 함께 기록 (README 설계 결정, docs/02_architecture.md)

결정	이유	대가 · 후속 조치
클럭: 단일(A) → 컨트롤러 1 : PHY 2(B). RTL UI 직렬화(C)는 채택하지 않음	핸드셰이크와 뱅크 타이밍을 먼저 닫고 CDC를 올림. UI 직렬화는 아날로그 PHY 경계 밖	클럭 하나와 CDC 리스크 → Gray FIFO, E12 클럭 대조표
컨트롤러는 Mini DRAM Controller v4를 파일 그대로, DFI 형태 신호로만 연결	그 저장소에서 600만 요청 랜덤 회귀와 독립 타이밍 체커로 검증. 이 칩은 경계와 칩 레벨 검증에 집중	고칠 일은 그 저장소에서 먼저(ModelSim 캐스트, q_empty). RD_PIPE 4가 PHY 왕복에 묶임
읽기 지연을 고정하지 않고 rddata_valid로 알림	RDQS 캡처 + Gray FIFO 지연이 RDQS 위상에 따라 한 클럭쯤 달라짐. 컨트롤러가 진행 중 RD를 셈	DFI tphy_rdlat 같은 계약 없음. 읽기 반환 FIFO는 왕복만큼 깊어야
TX 기어는 동기식(E1 엣지), RX는 RDQS → Gray FIFO만	clk_phy는 PLL에서 컨트롤러 클럭과 위상이 맞음 → 핸드셰이크 없이 매 클럭 한 워드	이전 토글 핸드셰이크 기어박스는 3클럭에 한 워드
레인 리페어는 shift 방식 (고장 핀부터 위 레인을 RED 쪽으로 한 칸)	핀마다 2:1 mux라 배선이 이웃 핀까지만. 치환 방식은 모든 레인에서 spare까지 경로	양 끝(PHY, DRAM)이 같은 설정을 가져야 함
리셋: CSR은 rst_sync_n, 데이터패스는 rst_sync_n & ~soft_rst	CSR까지 soft reset에 넣으면 soft_rst 비트가 다음 클럭에 스스로 지워짐	리셋 도메인이 둘
CSR 읽기 데이터를 레지스터로	칩 STA에서 cfg_addr → cfg_rdata 조합 feedthrough가 −0.174 ns	읽기 1클럭 늦음

클럭 구조 후보 (v0.1에서 비교)

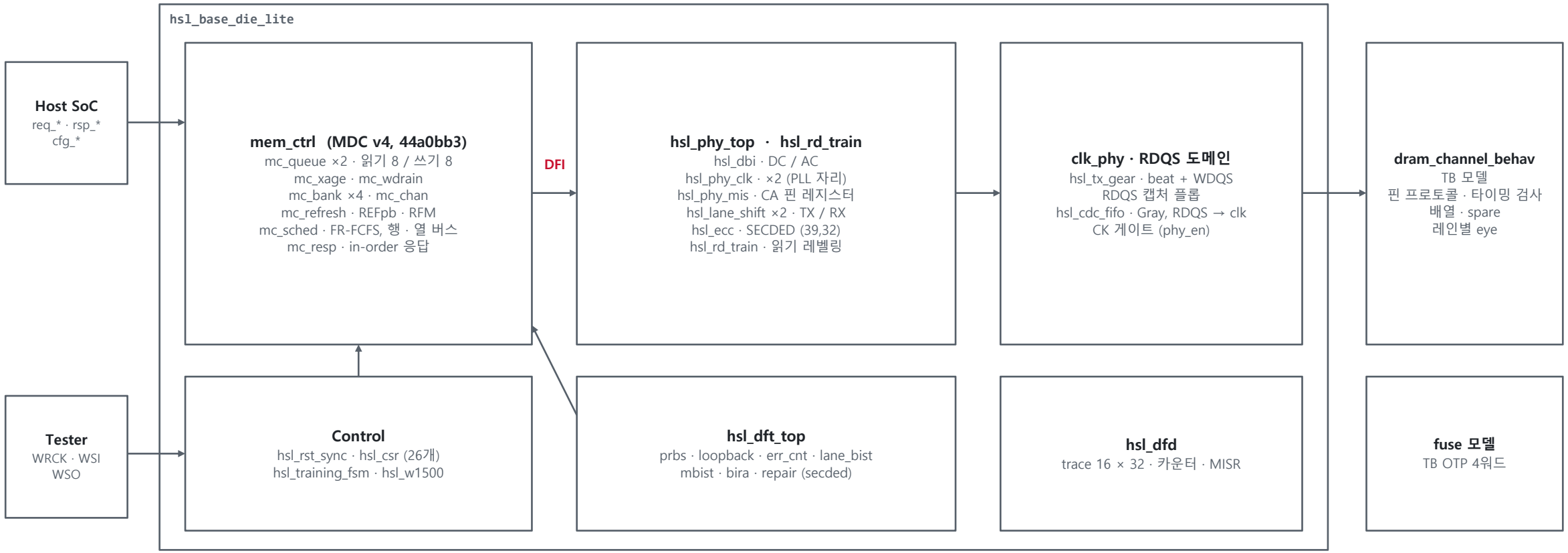
	구조	판단
A	단일 클럭, 버스트 단위	v0.1 출발점
B	컨트롤러 1 : PHY 2, 기어 + FIFO	E7에서 전환, 현재
C	RTL에서 UI 직렬화	채택 안 함 (아날로그)

경계를 먼저 정함

- 디지털: 핀 논리값 · enable, 핸드셰이크, CDC, 리셋, DFT 관측점
- 아날로그 · 풀커스텀: 드라이버, 샘플러, VREF, ZQ, 지연선, 클럭 스파인
- 핀 너머 DRAM은 TB의 채널 모델(dram_channel_behav): 프로토콜 · 타이밍 검사, 배열, spare, 레인별 eye
- custom HBM 전제: 컨트롤러가 base die에 있음. 표준 HBM(HBM3E까지)에서는 호스트 SoC 쪽

Top 구조: 컨트롤러 · PHY · DFT / DFD · Control

rtl/top/hsl_base_die_lite.sv · RTL 35개 모듈 · 클럭 세 개(clk 800 MHz, clk_phy 1.6 GHz, RDQS) · 이름은 RTL 인스턴스 그대로



- **포트**: req_* 14 bit 워드 주소. 요청 포트는 mission_mode && phy_en일 때만 열리고, 레인 BIST나 MBIST가 도는 동안은 그 엔진이 포트와 응답을 가짐. **핀**: CK · RCA 13 · CCA 10 · DQ / DBI / RED beat + ECC 2 · WDQS / RDQS. **사이드밴드**: rx_dly_code(지연선), rep_tab(spare), fz_*(fuse)
- **리셋**: CSR과 1500 포트는 rst_sync_n, 나머지는 rst_logic_n = rst_sync_n & ~soft_rst. deskew(0–4 tap)는 아직 경로에 없고 블록 테스트만

메모리 컨트롤러: Mini DRAM Controller v4를 DFI 형태로 연결

rtl/mc/ = 그 저장소 44a0bb3과 같은 파일 · 이전 in-order MC(hsl_mc_top · scheduler · bank FSM)는 0193c04에서 교체

모듈 (mem_ctrl)

모듈	역할
mc_queue ×2	읽기 8 · 쓰기 8, 큐 안의 나이 행렬, 주소 비교(RAW forwarding, WAR 보류)
mc_xage · mc_wdrain	두 큐 사이 나이 · 쓰기 배치(WR_HI 6 / WR_LO 2)
mc_bank ×4	열린 row, tRCD · tRP · tRAS · tRC · tRTP · tWR, auto-precharge, RAA
mc_chan	bank group별 tCCD · tRRD · tWTR (S / L), tFAW · tRTW · tRFC
mc_refresh	per-bank refresh: tREFI / 4마다 REFpb 하나, 부채 최대 4
mc_sched	행 명령 하나 + 열 명령 하나를 매 클럭, FR-FCFS, starvation guard, RFM
mc_resp	tag, 읽기 반환 FIFO(RD_PIPE 4), 쓰기 데이터 파이프, in-order 응답

검증은 두 층으로

- 컨트롤러 저장소: 랜덤 회귀 1,200회 · 600만 요청, 빌드 16개, 독립 bind 타이밍 체커(규칙 18 + 프로토콜 10), 참조 메모리 스코어보드 (MDC ab9d522)
- 이 칩: 채널 모델이 핀에서 프로토콜과 같은 뱅크 타이밍을 검사. tRCD + 1로 바꾼 변이 모델은 1,162건을 잡음 (0193c04)
- 스코어보드: 회귀 전체에서 호스트 요청 4,791개, 읽기 1,720개 비교, 불일치 0 (14fec6e)

DFI 형태 경계 (mem_ctrl ↔ hsl_phy_mis)

신호	방향	의미
dram_rcmd · rbank · row	MC → PHY	행 버스: ACT · PRE · REFpb. 다음 클럭 pin_rca
dram_ccmd · cbank · col · ap	MC → PHY	열 버스: RD · WR (+ AP). 다음 클럭 pin_cca
dram_wdata_en · wdata	MC → PHY	WR 뒤 WL 1 클럭
dram_data_valid · rdata	PHY → MC	지연 고정 없음, 순서만 지킴
q_empty	MC → DFT	큐가 빔: BIST가 핀을 거쳐 읽도록

핀에서 본 결과 (T2 · T3 · T12, 14fec6e)

- bank 1 명령 간격: ACT → WR 3, WR → PRE 4, PRE → ACT 3, ACT → RD 3 클럭. 모두 그 자리 규칙의 최소값
- idle 256클럭: bank마다 REFpb 4번 (tREFI 64)
- 랜덤 2,000 요청: 읽기 984개 일치, row hit 35%, 쓰기 큐에서 답한 읽기 15개, DFI 데이터 버스 32.9%
- WR 16 + RD 16 한 row: 이전 in-order MC 162 → 81 클럭
- 병목: RD를 4개(RD_PIPE)까지만 띄우는데 이 PHY 왕복이 8클럭 → 읽기 32개 97클럭

Mission PHY 데이터패스: CA 핀, 1.6 GHz beat, RDQS 캡처

hsl_phy_top · hsl_phy_mis · 컨트롤러 클럭마다 버스트 하나(4 UI × DQ 8) = DQ당 3.2 Gb/s · UI 직렬화와 지연선은 아날로그 쪽 (6a5a4ce)

한 클럭의 흐름

```
cycle t : 컨트롤러가 행 명령과 열 명령을 고름 → t+1에 pin_rca, pin_cca (한 클럭씩)
WR      : t+1 wrdata_en (WL 1) → DBI → lane shift(4 심볼) → ECC 인코드 → tx_word − E1 −▶ beat0 (UI0, UI1) − E0 −▶ beat1 (UI2, UI3) + WDQS
RD      : DRAM − RL −▶ beat0, beat1 on RDQS −▶ 캡처 플롭(RDQS) −▶ Gray FIFO −▶ clk −▶ unshift −▶ DBI 복원 −▶ 레지스터 −▶ ECC 디코드 −▶ rddata_valid
loopback: RD를 핀으로 내지 않고 마지막 쓰기 버스트(lb_hold)로 답함
```

핀 (hsl_base_die_lite 포트)

핀	폭	내용
pin_ck	1	clk_phy를 ck_en으로 게이트 (phy_en일 때만)
pin_rca	13	{cmd 3, bank 2, row 8}
pin_cca	10	{cmd 3, bank 2, col 4, ap}
DQ beat	10 핀 × 2 UI	{RED, DBI, DQ[7:0]}, clk_phy beat 둘
ECC	2 핀 × 2 UI	검사 비트 7 + 0, DBI · 리페어 밖
WDQS · RDQS	1 · 1	쓰기 '01' 패턴 · 읽기 = CK + tDQSCK

클럭과 경계

- TX 기어는 동기식: 컨트롤러 엣지 반 주기 뒤 PHY 엣지(E1)에서 워드를 받아 beat 두 개로. E1은 컨트롤러 토글을 PHY 클럭으로 비교해 찾음(FF 1개)
- RX: 채널 모델이 RDQS 하강 엣지에 beat를 내고 RDQS 상승 엣지에 캡처. 40 → 48 bit 깊이 4 Gray FIFO로 clk에 넘김
- CK는 ck_en(clk_phy 하강 엣지에서 phy_en 샘플)으로 게이트: mission 밖에서 0

PHY 블록 면적 (14fec6e 칩 넷리스트, μm²)

블록	면적
phy_mis · cdc_fifo · tx_gear	3,819 · 1,686 · 395
ecc · lane_shift ×2 · dbi · phy_top	1,960 · 354 · 320 · 306
PHY digital 합계 (FF 910)	8,888 (17%)

리셋, 트레이닝 FSM, 요청 포트 게이팅

비동기 assert · 2-FF 동기 deassert · FSM은 done 펄스를 기다림 · 요청 포트는 MISSION + phy_en, 테스트 엔진이 돌면 엔진 차지

rtl/top/hsl_rst_sync.sv · hsl_base_die_lite.sv

```
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) sync_r <= 2'b00;
  else        sync_r <= {sync_r[0], 1'b1};  // 해제는 2클럭 뒤
assign rst_no = sync_r[1];
assign rst_logic_n = rst_sync_n & ~soft_rst;  // CSR · 1500은 rst_sync_n
```

rtl/ctrl/hsl_training_fsm.sv

```
TR_RESET:   if (train_start) state <= TR_CKE;
TR_CKE:     state <= TR_MRS;
TR_MRS:     state <= TR_ZQCAL;
TR_ZQCAL:   if (zq_done)     state <= TR_WRLVL;
TR_WRLVL:   if (wrlvl_done)  state <= TR_RDLVL;
TR_RDLVL:   if (rdlvl_done)  state <= TR_DESKEW;
TR_DESKEW:  if (deskew_done) state <= TR_WRTRAIN;
TR_WRTRAIN: state <= TR_MISSION;
assign cke = (state != TR_RESET);  // CKE는 RESET에서만 0
```

- **TB는 리셋을 negedge에서 해제**: posedge와 같은 시각이면 첫 싱크 플롭과 레이스가 생겨 1클럭 빨리 풀림 (이슈 B, edb139a)
- **train_bypass**가 case보다 우선: 1클럭 뒤 MISSION. RDLVL 상태는 아직 done 펄스를 기다리고, 읽기 레벨링은 MISSION에서 CSR로 돌림

rtl/top/hsl_base_die_lite.sv · 요청 포트

```
assign port_open    = mission_mode && phy_en;
assign mc_req_valid = port_open && (tp_busy ? tp_req_valid : req_valid);
assign mc_req_addr  = tp_busy ? tp_req_addr : req_addr;
assign mc_rsp_ready = tp_busy ? 1'b1 : rsp_ready;
assign req_ready    = port_open && !tp_busy && mc_req_ready;
assign rsp_valid    = mc_rsp_valid && !tp_busy;
// tp_* = 레인 BIST 또는 MBIST (hsl_dft_top이 하나만 고름)
```

왜 이렇게 막는가

- PHY가 꺼져 있으면 RD가 응답을 받지 못함 → 포트 자체를 닫음. phy_en · 루프백은 컨트롤러가 idle일 때만 바꿈
- 엔진이 도는 동안 응답을 엔진이 모두 가져감(rsp_ready 1). 호스트는 조용하고 응답을 다 받은 상태에서 시작해야 함(칩은 검사하지 않음)
- 두 엔진의 start가 겹치면 잡아 두었다가 포트가 비면 냄, 레인 BIST 먼저. 버리면 읽기 레벨링이 BIST busy를 기다리다 멈춤
- SMOKE: MISSION이어도 phy_en = 0이면 포트 닫힘, 응답을 막으면 요청 15개(tag 수)에서 ready 0, soft reset 뒤 복귀

CA 핀 레지스터와 동기 2:1 TX 기어

명령은 한 클럭 레지스터 뒤 핀으로 · 워드는 E1 엣지에서 받아 beat 두 개로, 핸드셰이크 없이 매 클럭 한 워드 (6a5a4ce)

rtl/phy/hsl_phy_mis.sv · CA 핀

```
// 행 · 열 버스를 한 클럭 레지스터로 (코드는 dram_cmd_e 그대로)
rca <= {rcmd, rbank, row};        // 13 bit
cca <= {ccmd, cbank, col, ap};    // 10 bit, row 없음
// RD 추적: 핀으로 나간 RD마다 그 bank의 열린 row로 호스트 주소
if (cca[CCA_W-1 -: 3] == 3'(CMD_RD))
  rdq[rdq_w] <= host_addr(bank, open_row[bank], col);
```

rtl/phy/hsl_tx_gear.sv

```
always_ff @(posedge clk_ctrl) ctog <= ~ctog;     // 컨트롤러 토글
wire mid = (ctog != ctog_q);                     // 이 PHY 엣지가 E1
always_ff @(posedge clk_phy) begin
  ctog_q <= ctog;
  if (mid) begin dout <= din_valid ? din[W/2-1:0] : '0;   // beat0
                 hi   <= din[W-1:W/2]; hi_v <= din_valid; end
  else      begin dout <= hi_v ? hi : '0; hi_v <= 1'b0; end // beat1
end
```

동기 기어로 바꾼 이유

- 이전 토글 핸드셰이크 기어박스(hsl_gearbox): req · ack 2-FF 왕복 때문에 3클럭에 한 워드 → 목표 대역 경로가 아님
- clk_phy는 PLL에서 clk와 위상이 맞으므로, 반 주기 뒤 PHY 엣지에서 받으면 고정 지연 한 경로. STA는 clk → clk_phy 반 주기로 봄
- 쓰기 beat가 없으면 0을 내 TX 핀이 쉼 (전력 규칙)

결과

항목	값
E7 블록: 워드 2개 → 연속 beat 4개	C3D4 · A1B2 · 7788 · 5566
T12: WR 16 + RD 16 (bypass framer 대비)	bypass 207 → mission 162 클럭 (6a5a4ce)
같은 트래픽, v4 컨트롤러	81 클럭 (ffc073d)
hsl_tx_gear 면적	325 µm² · FF 44 (블록, 6a5a4ce)
clk → clk_phy 반 주기 경로	ECC 인코더 포함 +0.044 ns (14fec6e)

RDQS → clk Gray FIFO와 크로싱 제약

포인터는 소스 클럭의 Gray 플롭에서만 넘김 · 크로싱은 false path가 아닌 set_max_delay · 비동기 clock group 쓰지 않음

rtl/phy/hsl_cdc_fifo.sv

```
// wgray (clk_w) -> wptr_g_s1 -> wptr_g_s2 (clk_r)
always_ff @(posedge clk_w ...)
  wbin  <= wbin_next;
  wgray <= bin2gray(wbin_next);   // 넘어가는 유일한 쓰기 쪽 값
always_ff @(posedge clk_r ...)
  wptr_g_s1 <= wgray;             // crossing
  wptr_g_s2 <= wptr_g_s1;
assign rd_empty = (rgray == wptr_g_s2);
assign wr_full  = (wgray == {~rptr_g_s2[AW], ~rptr_g_s2[AW-1],
                             rptr_g_s2[AW-2:0]});
```

syn/constraints/hsl_base_die_lite.sdc

```
create_clock -name rdqs -period 0.625 [get_ports pin_rdqs]
# 소스 Gray 플롭 -> 첫 싱크 플롭: 빠른 쪽 주기
set_max_delay 0.625 -from u_phy/u_mis/u_rx_fifo/wgray* \
                    -to   u_phy/u_mis/u_rx_fifo/wptr_g_s1*
# FIFO 데이터 -> clk
set_max_delay 1.25 -from [FIFO mem] -to [clk 도메인]
```

규칙과 그 이유

- 조합 XOR(bin2gray) 출력이 넘어가면 여러 비트가 바뀌는 순간의 glitch를 첫 싱크 플롭이 잡을 수 있음 → Gray는 플롭에서 (리뷰, d1f8e2b)
- false path는 그 경로의 지연을 STA가 보지 않게 함 → 한 Gray 단계의 비트가 한 주기 안에 닿도록 set_max_delay
- 비동기 clock group은 그 max_delay를 덮어 false path로 만들므로 쓰지 않음. RDQS 도메인을 떠나는 경로는 FIFO뿐

칩 STA (14fec6e, OpenSTA 3.1, 이상 클럭)

검사	결과
Gray 포인터 크로싱	0.104 ns / 0.625 ns
FIFO 데이터 → clk	0.401 ns / 1.25 ns
rdqs 도메인 (pin_rx_dv → 버스트 레지스터)	slack +0.106 ns
T6 블록: 두 클럭 사이 3 word	순서 유지, rd_empty 정상

- **STA 흐름 문제**: 합성 플롭 이름이 SDC 패턴과 달라 max_delay가 아무 셀도 잡지 못했음(경고만) → 레지스터 이름 부여 (8af801b, p.19)

DBI, shift 방식 레인 리페어, 레인 BIST

DBI-DC / AC는 CTRL[6]로 선택(p.21) · 논리 레인 9개 {DBI, DQ}가 핀 10개 {RED, DBI, DQ}를 씀 · BIST는 컨트롤러 요청 포트로 (0cae795)

rtl/phy/hsl_lane_shift.sv (고장 핀 k)

```
//   P[i] = L[i]     i < k
//   P[k] = 0
//   P[i] = L[i-1]   i > k      (RED가 L[8] = DBI를 받음)
// RX: L[i] = (i < k) ? P[i] : P[i+1]
assign tx_p[i] = (k > i) ? tx_l[i] :
                 (k < i) ? tx_l[i-1] : 1'b0;   // 핀마다 2:1 mux
assign rx_l[i] = (k > i) ? rx_p[i] : rx_p[i+1];


lane 3 고장   DQ0 DQ1 DQ2 DQ3 DQ4 DQ5 DQ6 DQ7 DBI RED
              L0  L1  L2  --  L3  L4  L5  L6  L7  L8
```

레인 BIST (hsl_lane_bist)

- 요청 포트로 한 bank · row의 컬럼 0–15에 PRBS7 16워드 쓰기 → q_empty까지 기다림 → 다시 읽어 DQ 레인별 비교, sticky 기록
- q_empty를 기다리는 이유: 쓰기 큐에 남은 주소를 읽으면 컨트롤러가 큐에서 답해 링크를 거치지 않음 (통합 이슈, 0193c04)
- 주소는 CA 핀으로 가므로 DQ 고장이 주소에 섞이지 않음
- 한계: DBI 핀 고장은 DBI off에서 양방향 반전이 상쇄돼 보이지 않음 → 경계 스캔 필요(1500 WBR 없음)

T10: 검출 → 재매핑 → 재검사 (14fec6e)

단계	하는 일	결과
검출	LANE_BIST start, 링크 핀에 stuck-at 주입	16 / 16 (DQ 8 × 0 / 1), 정확히 그 레인
재매핑	REMAP에 그 핀, DRAM 쪽도 같은 설정	
재검사	BIST 다시	16 / 16 PASS
트래픽	lane 5 stuck-at-1 리페어 + DBI-AC	쓰기 · 읽기 16개 일치
한계	spare 하나: lane 2 · 6 동시 고장	에러 0x44, 단일 비트 0

면적 (블록, Nangate45 typical)

블록	μm²	FF	지연
hsl_lane_shift (TX + RX)	61	0	0.227 ns
이전 치환 방식 × 4 beat (TX만, 4d38ae9)	143	0	0.177 ns
hsl_lane_bist (0193c04)	630	54	0.740 ns
hsl_dbi DC + AC mux 체인 (e30ea96)	289	0	0.882 ns

- **T7 핀 값**: lane 3이면 FF → 핀 1F7(DQ3 쉼, DBI 핀이 DQ7, RED가 DBI), lane 8이면 DBI가 RED로(200)

읽기 레벨링: 지연선 코드 스윕과 eye 중앙

디지털은 레인별 지연선 코드(0–31, rx_dly_code)만 내고 지연선 · 샘플러는 아날로그 쪽 · 판정은 레인 BIST 재사용 (381dd5b)

rtl/phy/hsl_rd_train.sv

```
// 모든 레인에 같은 코드 c = 0 … 31, 코드마다 레인 BIST 한 번
if (lane_pass[li]) begin
  if (cur_len[li] == 0) cur_lo[li] <= code;
  cur_len[li] <= cur_len[li] + 1;
  if (cur_len[li] + 1 > best_len[li]) begin
    best_lo[li]  <= (cur_len[li] == 0) ? code : cur_lo[li];
    best_len[li] <= cur_len[li] + 1; end
end else cur_len[li] <= 0;
// 끝: trained = lo + (len - 1) / 2, len < 2인 레인이 있으면 ok = 0
```

채널 모델의 eye

- 레인마다 eye [lo, hi]: 코드가 안이면 자기 UI, 앞이면 이전 UI, 뒤면 다음 UI를 읽음 (버스트 밖은 0)
- 판정을 레인 BIST에 맡겨 이 블록은 스윕과 구간 추적만 함. 32 코드 × BIST 한 번
- MISSION에서 CSR RDTRAIN으로 시작(요청 포트가 MISSION에서만 열림). 쓰기 방향 레벨링은 없음

T11 결과 (14fec6e, 고정 eye 회차)

레인	eye [lo, hi]	학습 코드	기본 코드 16에서
0	[3, 9]	6	BIST 실패
1	[9, 17]	13	통과
2	[18, 25]	21	BIST 실패
3	[6, 17]	11	통과
4	[20, 25]	22	BIST 실패
5	[1, 10]	5	BIST 실패
6	[12, 16]	14	통과
7	[15, 25]	20	통과

- **학습 전**: BIST가 정확히 레인 0 · 2 · 4 · 5(0x35)를 실패로 표시. **학습 뒤**: BIST PASS, 호스트 트래픽 읽기 16개 일치
- **랜덤 eye 회차**도 8레인 eye 시작 · 폭 · 중앙 코드가 정확. 닫힌 eye는 ok = 0

hsl_rd_train (Nangate45)	μm²	FF	최대 경로
블록 (381dd5b)	2,551	228	0.707 ns
칩 넷리스트 (14fec6e)	2,596	228	

검증 환경과 테스트 목록

tb_hsl_smoke(칩 + 채널 모델) 안에서 tb_hsl_blocks(블록 인스턴스)까지 한 번에 · [ALL] PASS 외에는 non-zero exit

ID	대상	확인 내용
SMOKE	칩 기본	ID · VERSION · TIMING, 미정의 주소 0, 리셋 직후 핀 · ready 0, phy_en 없으면 포트 닫힘, tag 15에서 ready 0, soft reset
T1	트레이닝 FSM	순서 진행, 대기 상태는 done 펄스 전까지 머묾
T2	핀의 DRAM 명령	행 · 열 CA 코드와 간격이 규칙 최소값, idle 256클럭에 bank마다 REFpb 4번
T3	랜덤 트래픽	요청 2,000개(읽기 · 쓰기 무작위, 응답은 4클럭 중 3클럭만), 읽기 984개 일치
T4	posted · forwarding	쓰기는 다음 클럭 응답, 바로 뒤 읽기는 쓰기 큐에서 (핀에 RD 없음)
T5 · T9	DBI DC · AC	핀 beat 값, 직렬 참조 모델 · 이항분포 기대값, 읽기 DBI 복원
T6 · E7	CDC FIFO · TX 기어	두 클럭 사이 3 word 순서, 워드 두 개 → 연속 beat 넷
T7 · T10	레인 리페어 · BIST	shift 핀 값, stuck-at 16/16 검출 · 복구, 두 레인 수리 불가
T8	PRBS · 루프백	ERRCNT, 루프백 RD가 마지막 쓰기 버스트를 돌려줌
T11 · T12	읽기 레벨링 · mission	eye 중앙 코드, 지연 · 대역 측정, 채널 모델 오류 0
T13–T17	DFT / DFD	MBIST · repair · ECC · DFD · 1500 포트 (p.23–28)
E9 · E11	deskew · PRBS seed	tap 0 통과 · tap 2 지연, seed 5A 첫 비트 1 · seed 0 무시

tb_hsl_smoke 구조

```
dut       hsl_base_die_lite (수정 없음)
u_dram    dram_channel_behav
          핀 프로토콜 · 같은 뱅크 타이밍
          쓰기 데이터 전 RD · WDQS 검사
          배열 · spare · eye · 고장 주입
ref mem   호스트 응답 스코어보드
fuse      OTP 모델 (리셋에도 유지)
u_blocks  tb_hsl_blocks (블록 인스턴스)
```

합격 규칙

- 체크마다 PASS / FAIL, 하나라도 실패면 [ALL] FAIL (n)과 non-zero exit. 타임아웃도 실패
- 비교는 !== · ===: X나 Z는 실패
- 채널 모델 오류 카운터(프로토콜 · 타이밍 · 쓰기 전 RD · WDQS)는 끝에서 모두 0이어야
- Verilator 5.052와 ModelSim-Altera 10.1d에서 같은 회귀

회귀와 커버리지 결과

23 / 23 PASS (Verilator 5.052 · ModelSim-Altera 10.1d) · Line 95.5% · Branch 91.4% · Toggle 78.8% · commit 14fec6e

sim/regress.log (Verilator, 발췌)

```
[T7] unit PASS      [E9] PASS
[E11] prbs PASS     [E7] PASS
[T6] PASS           [SMOKE] PASS
[T9] unit PASS      [T1] PASS
[T2] bank 1 at the pins:
  ACT +3 WR +4 PRE +3 ACT +3 WR
[T2] REFpb per bank 4 4 4 4
[T3] 2000 requests (984 reads)
  row hits 35.0%
[T4] [T5] [T7] [T8] [T9] PASS
[T10] found 16/16, repaired 16/16
[T11] [T12] [T13] [T14] PASS
[T15] [T16] [T17] PASS
[T12] channel model: errors
  protocol 0, timing 0, WDQS 0
[SB] 4791 host requests,
  1720 reads compared
[ALL] PASS
```

모듈별 커버리지 (rtl/ 35개, 인스턴스 합산)

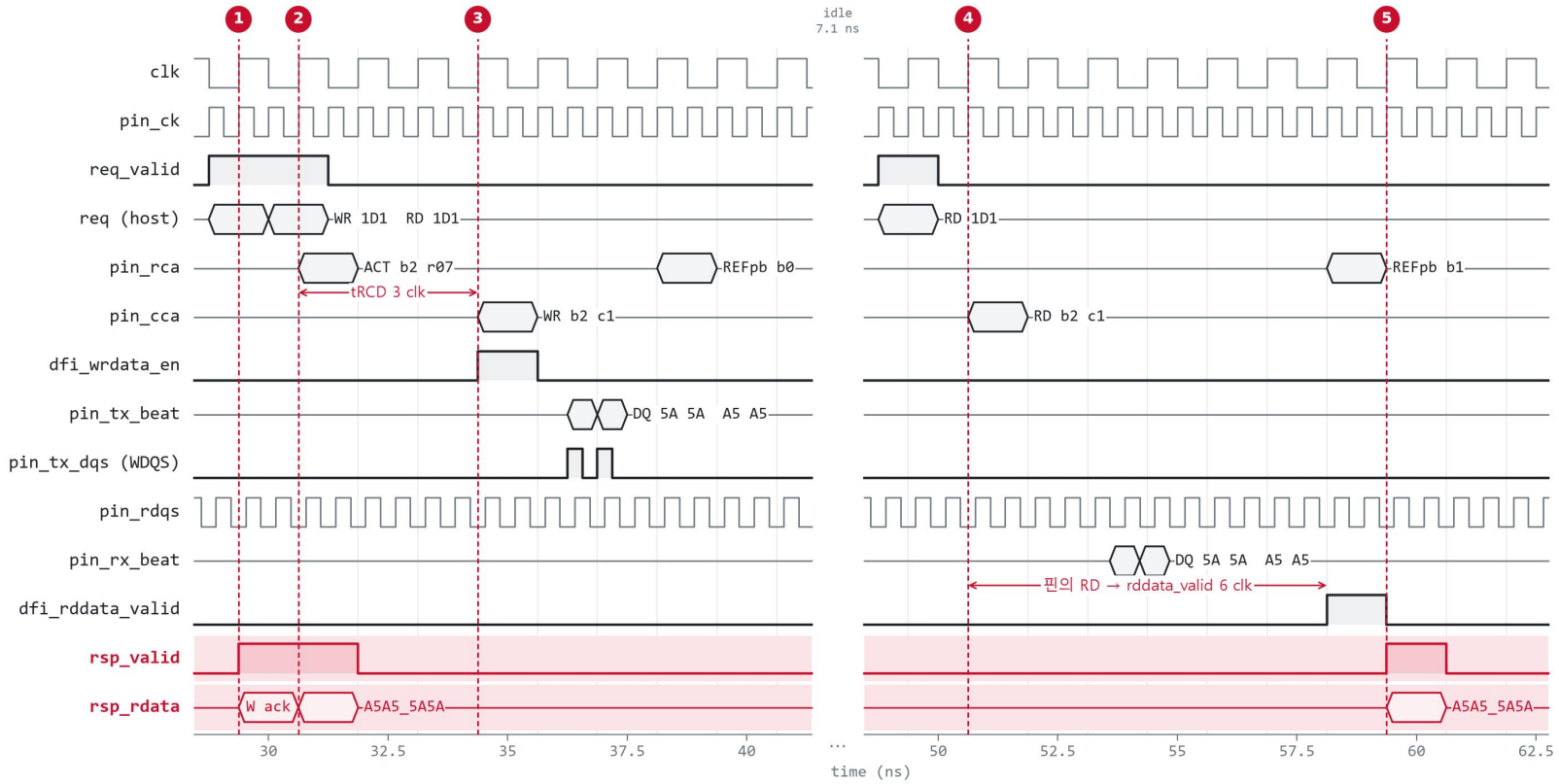
모듈	Line	Branch	Toggle
mem_ctrl · queue	100	100	86.3 · 81.1
mc_bank	100	88.5	86.2
mc_sched	91.9	62.5	81.5
mc_resp	80.0	100	100
phy_mis · ecc	100	100	77.1 · 78.3
cdc_fifo	100	100	66.2
dbi	100	75.0	100
rd_train	93.3	90.9	70.3
csr · w1500	98.1 · 100	100 · 95.8	66.3 · 68.6
mbist · bira	96.9 · 91.2	89.1 · 90.6	82.3 · 87.2
dfd	75.0	96.7	69.0
deskew · err_cnt	100 · 66.7	83.3 · 100	23.4 · 23.8
전체	95.5	91.4	78.8

닿지 않은 곳

- 컨트롤러 branch 구멍 대부분은 이 칩이 쓰지 않는 파라미터 경로(all-bank refresh, close page, 버스 하나, FCFS). 컨트롤러 저장소가 빌드 16개로 돔
- 실제 구멍: starvation 우선(160클럭), RFM, refresh 부채 포화, err_cnt clear 우선순위
- dfd line 75%: 쓰지 않은 trigger(UE · PRBS · MBIST · bank 명령)
- deskew · err_cnt toggle 23%: 블록 테스트만, 카운터 상위 비트 미도달
- 이전 in-order MC(4feb39f) 87.5 / 96.6 / 56.6% → T3 랜덤 주소 · 데이터로 toggle 상승

T4 파형: posted write, 쓰기 큐 forwarding, 핀의 RD와 RDQS 반환

Verilator VCD, commit 0193c04 (가운데 idle 7.1 ns 생략) · 신호 이름은 RTL 계층 그대로



- ① 쓰기 요청(호스트 1D1 = bank 2, row 07, col 1)을 받은 다음 클럭에 응답 (posted)
- ② 바로 뒤 같은 주소 읽기는 쓰기 큐에서 A5A5_5A5A로 답하고 pin_cca에 RD 없음. 같은 클럭 ACT b2 r07
- ③ tRCD 3클럭 뒤 WR b2 c1, 다음 클럭 wrdata_en, TX beat 둘(5A 5A A5 A5)과 WDQS. 사이에 REFpb b0 · b1
- ④ 쓰기가 나간 뒤 다시 읽으면 RD b2 c1이 핀으로
- ⑤ RDQS beat 둘이 Gray FIFO를 지나 핀의 RD 6클럭 뒤 rddata_valid, 요청 9클럭 뒤 응답 (ECC 단계 전, 지금은 7 · 10)

핀에서 잰 지연 · 대역과 남은 병목

T2 · T3 · T12 · 채널 모델 카운터 (commit 14fec6e, ECC 읽기 단계 포함) · 이전 in-order MC 수치는 같은 트래픽으로 비교

항목	지금	이전 · 비고
WR 16 → RD 16 한 row (첫 요청 → 마지막 응답)	81 클럭	in-order MC 162 (8af801b 77)
열린 row 읽기 하나: 요청 → 응답	10 클럭	ECC 단계 전 9
핀의 RD → rddata_valid	7 클럭	ECC 단계 전 6
읽기 32개, bank group 두 개	97 클럭	ECC 단계 전 76
랜덤 2,000 요청: DFI 데이터 버스 사용	32.9%	6,033 클럭, row hit 35%
핀 명령 간격 ACT → WR · WR → PRE · ACT → RD	3 · 4 · 3	규칙 최소값

```
[T12] 채널 모델 카운터, 회귀 전체
1616 ACT, 1611 PRE, 3245 RD, 5096 WR, 1457 REFpb
errors: protocol 0, timing 0, read before its write data 0, WDQS 0
[T3] at the pins: 1291 ACT, 1290 PRE, 969 RD, 1016 WR, 376 REFpb
     row hits 35.0%; 15 reads answered from the write queue
[T2] PRE +11 ACT: 그 사이 bank 1 REFpb (tRFCpb 8)가 끼어서
```

병목: 진행 중인 RD 수

- 컨트롤러는 RD를 4개(RD_PIPE)까지 띄우고, RD 하나는 낸 클럭부터 data_valid까지 슬롯을 잡음
- 컨트롤러 저장소 모델에서는 3클럭, 이 PHY에서는 8클럭(ECC 단계로 한 클럭 더) → 읽기만으로는 데이터 버스 절반 아래
- ECC 디코더 앞 레지스터 단이 한 클럭을 더함 → 76 → 97
- 해결: 읽기 반환 FIFO(RD_PIPE)를 PHY 왕복만큼 (컨트롤러 저장소에서). 큐 · 스케줄러는 그대로
- 800 MHz 문제는 별개: 컨트롤러가 459 MHz에서 닫힘 (p.19)

이슈 기록: 원인과 조치

실습 중 고친 문제(E) · 재검증에서 찾은 문제(A–E) · 통합 · STA · 리뷰 · 측정 · 회귀 테스트를 고친 항목마다 추가

ID	증상	원인	조치
E1	soft reset 유지 불가	CSR까지 soft reset에 넣어 CTRL bit 0이 다음 클럭에 지워짐	CSR은 rst_sync_n만
E8	CDC 포인터 경로가 타이밍에서 빠짐	false path는 그 경로 지연을 보지 않음	set_max_delay, false path 금지
E11	고장이 통과하거나 정상처럼 보임	seed 0끼리 비교, 16 bit wrap-around	seed 0 무시, 카운터 포화
E12	PHY 주기 1248 ps · I/O delay 4.0 ns	반주기 정수 312 ps · 예전 10 ns 클럭의 40%	#312.5ps, 0.5 ns, 4곳 대조표
A · B	312.5 ps → 313 ps · 리셋 1클럭 빠름	timescale 1 ps · posedge 해제 레이스	100 fs 정밀도, negedge 해제 (edb139a)
D	큐가 멈춘 채 풀리지 않음	REF · 예약 opcode가 head에 오면 head_ok = 0	컨트롤러 교체로 해소 (0193c04)
E	리셋 없는 플롭 200개	큐 데이터 배열이 리셋 분기 밖	배열 분리, DFT 규칙 명시 (edb139a)
리뷰	Gray glitch 위험 · 루프백 RD · ModelSim	조합 Gray, 제약이 크로싱을 비켜 감, BANK_W'(gi) 캐스트	Gray 플롭, 제약 이동, lb_hold, localparam (d1f8e2b)
통합	가져온 컨트롤러가 ModelSim에서 실패	같은 파라미터 폭 캐스트 21곳	컨트롤러 저장소에서 고쳐 회귀 후 가져옴 (44a0bb3)
통합	레인 BIST가 링크를 안 거칠 수 있음	쓰기 큐 주소 읽기는 큐에서 답함	q_empty 출력, 큐가 빈 뒤 읽기 (0193c04)
STA	1.6 GHz 도메인이 분석에서 빠짐	PLL 빈 쉘 → 생성 클럭이 마스터와 끊김	PLL 타이밍 스텁 (8af801b)
STA	CDC set_max_delay가 아무것도 안 묶음	합성 플롭 이름 ≠ SDC 패턴 (경고만)	레지스터 이름 부여 (8af801b)
측정	greedy BIRA가 수리 가능한 맵을 놓침	must-repair 뒤 greedy 선택이 뒤를 막음	spare 순서 탐색 (6c29d60)
리뷰	MBIST 중 읽기 레벨링 멈춤	다른 엔진 busy 중 start를 버림	start 보류, 레인 BIST 우선 (f018966)

칩 면적: 블록별 분포 (Nangate45, 배선 없음)

bash syn/sta_chip.sh · Yosys 0.52가 모듈 경계를 유지해 합성, ABC가 1.25 ns 목표로 Nangate45 typical 매핑, 팬아웃 버퍼 트리 (commit 14fec6e)

블록	µm²	Cell	FF	비중	구성 (µm²)
Memory Controller	23,570	13,229	2,004	44%	쓰기 큐 5,431 · 읽기 큐 4,633 · resp 6,074 · sched 3,721 · bank ×4 1,372 · chan 690 · xage 608
PHY digital	8,888	4,146	910	17%	phy_mis 3,819 · ecc 1,960 · cdc_fifo 1,686 · tx_gear 395 · lane_shift ×2 354 · dbi 320
읽기 레벨링	2,596	1,475	228	5%	레인당 구간 추적 상태 27 FF × 8
DFT	8,926	4,623	794	17%	w1500 1,369 · mbist 2,836 · bira 1,972 · repair 1,144 · prbs 496 · lane_bist 417
SECDED 코덱	1,074	785	0	2%	인코더 ×3 (fuse 1, ECC 2) · 디코더 ×2
DFD	6,023	2,358	701	11%	trace 16 × 32 bit 플롭, 카운터 4개, MISR
CSR	1,993	1,021	165	4%	주소 26개, 읽기 mux와 읽기 데이터 레지스터
기타	247	144	6	0.5%	top (CSR 버스 mux) · training_fsm · rst_sync
합계	53,316	27,781	4,808		

면적 변화 (같은 흐름, 측정 commit)

commit	µm²	FF	더한 것
8af801b	32,184	2,801	STA 흐름, v4 MC
6c29d60	39,225	3,351	MBIST · BIRA · fuse
14fec6e	53,316	4,808	ECC · DFD · 1500

읽는 법

- 컨트롤러 저장소는 mem_ctrl만 flatten 합성해 18,912 µm². 여기서는 블록별로 나누려고 경계를 유지해 그만큼 큼
- ABC 실행 간 차이로 같은 RTL도 0.6% 안팎 움직임. SRAM 매크로 없음: 큐 · 응답 버퍼 · trace 모두 플롭
- 전력(mW)과 사인오프 slack은 적지 않음 (라이브러리 · 배선 근거 없음)

칩 STA: 세 클럭과 CDC, 흐름에서 먼저 고친 것

sv2v → Yosys → fanout_buffer.py → OpenSTA 3.1 + 저장소 SDC · 이상 클럭, 배선 없음: 경로를 찾기 위한 것이지 사인오프가 아님 (14fec6e)

결과

클럭 · 검사	결과
clk_phy 1.6 GHz (0.625 ns)	+0.044 ns (ECC 인코더 반 주기)
rdqs (0.625 ns)	+0.106 ns
FIFO 크로싱 (set_max_delay)	Gray 0.104 / 0.625 · 데이터 0.401 / 1.25
clk 800 MHz, HSL 로직	모두 통과
clk 800 MHz, 컨트롤러	−0.930 ns · 2.18 ns = 459 MHz
hold · removal	−0.03 ns CSR 리셋 핀 (CTS 뒤 수리 대상)

숫자가 의미를 갖기 전에 고친 흐름 (8af801b)

증상	원인 → 조치
플롭 44개 클럭 없음	PLL 빈 쉘 → 타이밍 스텁 .lib (clk → clk_phy 아크)
CDC 제약이 셀을 못 잡음	플롭 이름 _1234_ → 레지스터 이름 부여
리셋 recovery −27.7 ns	팬아웃 ~2,700 외삽 → 부하 16개마다 버퍼 트리

- **교훈**: 실제 흐름에서도 이름이 안 맞으면 CDC가 경고 하나 남기고 조용히 제약 밖으로 빠진다

STA가 짚어 RTL을 고친 경로 (DFT · ECC 경로는 p.28)

경로	slack	조치
PRBS LFSR → 32단 XOR → DBI-AC → TX 워드	−0.136	PRBS 워드를 레지스터로 (REG_OUT)
cfg_addr → 읽기 mux → cfg_rdata	−0.174	읽기 데이터 레지스터
리셋 해제 → 클럭 게이트 enable	−0.049	enable 플롭 리셋 제거 (ICG 꼴)
CTRL · REMAP → DBI-AC → TX 워드	−0.011	multicycle 2 (idle 때만 바뀜)

컨트롤러가 800 MHz를 못 닫는 이유와 방향

- 임계 경로: 큐 bank → row hit · 후보 → refresh 차단 → FR-FCFS 선택 → ACT 결정 → 뱅크 타이머 갱신. 한 클럭에 고르고 모든 타이머를 갱신하는 루프(약 35단)
- 호스트 입력 → req_ready도 조합(쓰기 큐 forwarding, 읽기 큐 충돌 검사): −0.521 ns
- DFI 1 : 2: 400 MHz에서 클럭마다 명령 슬롯 둘. 실제 컨트롤러 방식, 지금 논리 깊이로 닫힘 → 컨트롤러 저장소에서
- 스케줄러 파이프라인: 다음 클럭 후보가 바뀌는 루프라 추측 발행이 필요. 호스트 경계 레지스터 슬라이스는 입력 feedthrough만 없앰

PD 인계: 제약, 배치, 클럭 · 리셋 가이드

docs/05_pd_guide.md · syn/constraints/hsl_base_die_lite.sdc · 숫자는 E12 대조표로 문서 · pkg · TB · SDC와 일치 확인

SDC (syn/constraints/hsl_base_die_lite.sdc)

제약	내용
clk	1.25 ns (800 MHz), 포트 clk
clk_phy	u_phy_clk 출력에 generated clock ×2 (0.625 ns), PLL 자리
rdqs	pin_rdqs에 0.625 ns. 떠나는 경로는 읽기 FIFO뿐, max_delay로 묶음
ck_out	pin_ck에 generated clock (게이트 거친 clk_phy)
I/O delay	컨트롤러 주기의 40% = 0.5 ns: req_* · rsp_* · CA 핀 · rx_dly_code · fz_* · rep_tab
CDC	Gray 0.625, FIFO 데이터 → clk 1.25 (false path 아님)
rst_n	false path (v0.1 단순화, 실제는 recovery / removal 확인)
준정적 설정	CTRL · REMAP → TX 워드 데이터 핀 multicycle 2

배치 · 클럭

- PHY 핀 로직(CA 레지스터, TX 기어, RDQS 캡처 플롭, lane shift)은 범프 쪽, CSR · FSM · DFT는 안쪽
- RDQS 캡처와 FIFO 쓰기 쪽은 RDQS 핀 가까이: rdqs slack +0.106 ns가 가장 작은 여유 중 하나
- SRAM 매크로 없음: 컨트롤러 큐 · 응답 버퍼(FF 2,004)와 trace는 플롭. 실제 칩이면 trace는 SRAM

리셋 · DFT 규칙

- 동기화된 리셋이 플롭 약 2,700개로: 배치 전 ideal network, P&R에서 트리. OpenSTA 3.1은 set_ideal_network를 적용하지 않아 흐름이 버퍼 트리를 넣음
- 제어 상태는 모두 리셋, 큐 데이터 배열만 리셋 분기 밖(DFT 규칙에 명시). 스캔 체인 · ATPG는 넣지 않음
- deskew 4 tap = 8 UI까지 레인 스큐 허용 (아직 경로 밖, 블록만)
- hold: CSR 리셋 핀 removal −0.03 ns, CTS 뒤 다시

DBI-AC 추가: 전이 규칙, beat당 2:1 mux 체인, off · DC · AC 비교

CSR CTRL[6]로 DC · AC 선택 · AC는 직전 UI에 나간 바이트 대비 DQ 전이가 4개를 넘으면 반전 → UI당 DQ 전이 ≤ 4 (commit e30ea96)

두 규칙과 beat 0의 기준

- **DC**: beat의 1이 4개 초과 → 반전. high 핀 수를 줄임 (high에서 종단 전류가 흐르는 I/O)
- **AC**: 직전 UI 대비 DQ 전이 4개 초과 → 반전. 전이 수를 줄임 (종단 없는 짧은 채널의 스위칭 전력 · SSO)
- **beat 0**: 바로 앞 UI와 비교. bypass framer에서는 WR 헤더(DQ[3:0] = col)가 그 UI

rtl/phy/hsl_dbi.sv — AC 체인을 mux 하나로

```
// 직렬: beat마다 XOR → popcount → 비교 → 반전이 이어짐
r[b]   = pop8(beat[b] ^ beat[b-1]);   // 4 beat 병렬, beat[-1] = prev
inv    = inv ? (r[b] < 4) : (r[b] > 4); // b-1 반전 시 전이 = 8 - r
ac_inv[b] = inv;
// e30ea96: AC_SERIAL = 1이면 직렬 구조 (블록 TB가 두 구조 등가 확인)
```

hsl_dbi · Nangate45 typical (syn/ppa_block.sh)

구조	면적	Cell	입력 → 출력 지연
DC만 (4d38ae9)	130 μm²	101	0.449 ns
DC + AC, mux 체인 (기본)	289 μm²	224	**0.882 ns**
DC + AC, 직렬 (AC_SERIAL = 1)	301 μm²	229	1.822 ns

직렬은 블록 하나로 컨트롤러 주기 1.25 ns를 넘는다. mux 체인은 0.37 ns를 남기지만 칩 경로(큐 head → PRBS 먹스 → DBI → 리페어)는 칩 STA에서 다시 확인.

UI당 전이 · high 핀 수 (DQ + DBI, T9 측정)

데이터 (측정 위치)	지표	off	DC	AC
PRBS7 · WR 254개 (칩 핀)	전이	4.01	4.08	**3.35 (−16%)**
	high	4.03	**3.31 (−18%)**	4.13
55 AA 55 AA · WR 254개 (칩 핀)	전이	7.00	7.00	**1.64 (−77%)**
랜덤 연속 스트림 (블록)	전이	4.00	4.17	**3.38**
40,000 UI	high	4.01	**3.27**	4.36

- **DC는 high를, AC는 전이를 줄인다**. 랜덤에 가까운 데이터에서 각 규칙은 다른 쪽 지표를 조금 늘린다
- **전이가 몰린 데이터에서 AC가 크다**. 55/AA에서 DC는 아무것도 못 하고(beat마다 1이 4개), AC는 DQ를 고정하고 DBI만 바꾼다
- **이론값과 일치**. 랜덤 스트림이 이항분포 기대값(off 4.00 · DC 4.16 · AC 3.37)과 ±0.05 안

T9 · 회귀 15/15 PASS (Verilator 5.052 · ModelSim 10.1d)

- **블록**: 직렬 참조 모델과 랜덤 4,000 버스트, 두 AC 구조 등가, 동점(전이 정확히 4) directed
- **칩**: AC 핀 값 정확 비교, PRBS7 · 55/AA 스트림에서 규칙 위반 0
- **읽기**: 채널 모델이 RD 데이터에 DBI(DC · AC)를 붙이고 RX가 복원, 32 RD 일치. 핀 DBI = 1 수신 구멍 해소

리뷰 반영: CDC Gray 크로싱 · SDC · 루프백 RD · ModelSim 빌드 (d1f8e2b)

그때의 수정 기록 · 지금 FIFO 코드와 SDC는 p.10 · 당시 Verilator 5.052와 ModelSim-Altera 10.1d 모두 15/15 PASS

① CDC: Gray 포인터를 플롭에서 넘김

- **문제**: 싱크러나이저가 binary 플롭의 XOR(조합 Gray)을 바로 받음. 여러 비트가 바뀌는 순간의 glitch를 첫 싱크 플롭이 잡을 수 있음
- **수정**: wgray · rgray를 소스 클럭 플롭으로 두고 그 플롭만 넘김. full / empty 타이밍은 그대로

rtl/phy/hsl_cdc_fifo.sv

```
// 4feb39f: 조합 Gray가 넘어감
assign wgray = bin2gray(wbin);

// d1f8e2b: 소스 클럭의 Gray 플롭
always_ff @(posedge clk_w ...)
  wbin  <= wbin_next;
  wgray <= bin2gray(wbin_next);

// clk_r 쪽 2-FF 싱크
wptr_g_s1 <= wgray;  // crossing
wptr_g_s2 <= wptr_g_s1;
```

② SDC: 크로싱 자체를 제약

- **문제**: set_max_delay가 한 클럭 안의 *_s1 → *_s2를 묶고 있어, 실제 크로싱(소스 → *_s1)은 제약 밖
- **수정**: 소스 Gray 플롭 → 첫 싱크 플롭에 빠른 쪽 주기 0.625 ns. 한 Gray 단계의 비트가 한 주기 안에 도착. false path 아님

syn/constraints/hsl_base_die_lite.sdc

```
# 4feb39f: 한 클럭 안의 경로
set_max_delay $ctrl_period \
  -from u_fifo/wptr_g_s1* \
  -to   u_fifo/wptr_g_s2*

# d1f8e2b: wgray (clk_phy) -> s1 (clk)
set_max_delay $phy_period \
  -from u_phy/u_fifo/wgray* \
  -to   u_phy/u_fifo/wptr_g_s1*
# rgray (clk) -> rptr_g_s1 (clk_phy)
#   같은 형태
```

③ 루프백 RD · ModelSim 빌드

- **루프백 RD**: 루프백 중 RD가 0이나 다음 큐 head의 데이터를 돌려줌(요구 5 미충족) → 마지막 루프백 WR 버스트를 lb_hold에 두고 RD로 반환
- **ModelSim 10.1d**: 폭이 파라미터인 캐스트 BANK_W'(gi)를 파싱 못 함 → 타입이 있는 localparam. 이전의 ModelSim 통과 기재를 바로잡음

d1f8e2b 회귀 (Verilator 5.052 · ModelSim 10.1d)

```
[T7] unit PASS      [SMOKE] PASS
[E9] PASS           [T1] PASS
[E11] prbs PASS     [T4] PASS
[T3] PASS           [T5] PASS
[T7] PASS           [T8] PASS
[T2] PASS           [E7] PASS
[T6] PASS           [T9] unit PASS
[T9] PASS
[ALL] PASS

// T8: 채널 모델에 다른 값이 있는 col을 읽어 lb_hold 반환 확인
```

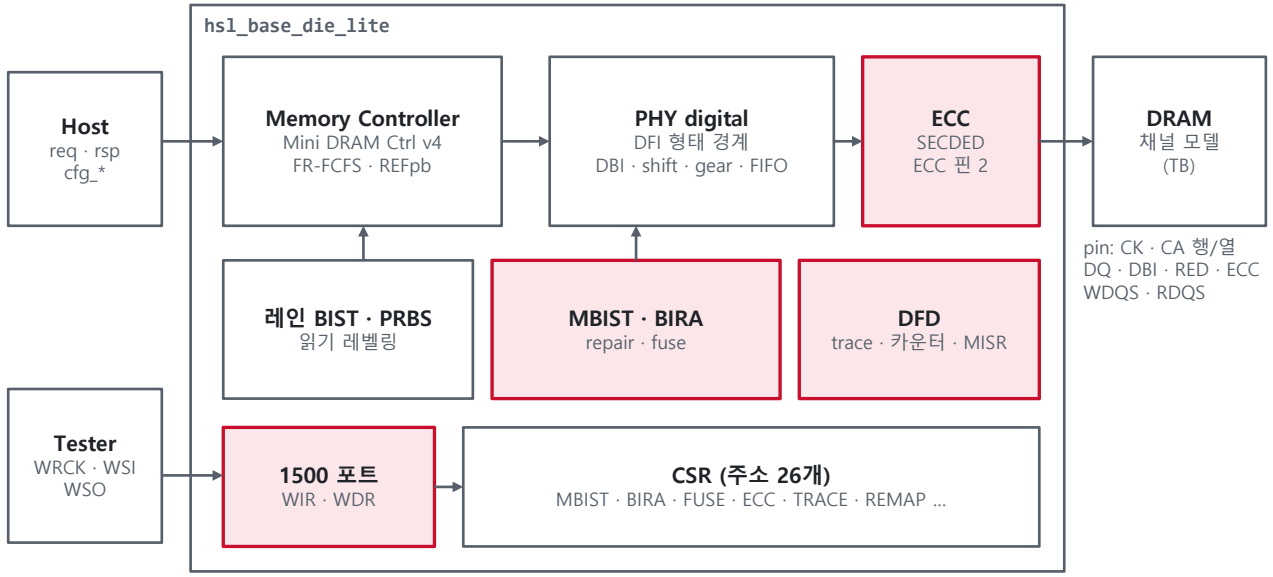
세 건의 공통점

모두 시뮬레이션은 통과한 상태에서 **설계 의도**(CDC 안전성, 제약이 걸리는 경로, 요구 5)와 **도구 호환성**을 따로 검토해 찾았다. 동작 검증과 별개로 코드 · 제약 리뷰 단계를 두고, 고친 항목마다 회귀 테스트를 추가했다(T8 루프백 RD).

DFT / DFD 확장: 배열 시험 · 수리 · ECC · 관측 · 시험 포트

빨간 테두리 = 이번에 더한 블록 · 시험 장비는 1500 포트 하나로 모든 단계를 돈다 (commit f018966 → 14fec6e, 쪽마다 측정 commit)

칩 구조 (14fec6e) · 컨트롤러 교체와 mission PHY 상세는 README



23 / 23 PASS
SMOKE · T1–T17 · E7 · E9 · E11, Verilator 5.052 · ModelSim 10.1d

53,316 µm² · 4,808 FF
Nangate45, 배선 없음 · DFT 17% · DFD 11% · ECC 4%

Line 95.5% · Branch 91.4%
Toggle 78.8% · rtl/ 전체, 인스턴스 병합

HSL 로직 STA 통과
800 MHz · 1.6 GHz · RDQS. 컨트롤러만 459 MHz

시험 흐름 (docs/08_test_flow.md) · 단계마다 회귀 테스트 하나

#	단계	합격 조건	실패하면
0	전원 · 리셋	fuse 4워드 SECDED 디코드 → repair 표, 정정 불가 0	fuse bin
1	접근	1500 ID HSL1 · VERSION 0x300	접근 bin
2	디지털 데이터패스	PRBS7 병렬 루프백 오류 0	로직 bin
3	링크 핀	레인 BIST. 한 레인 실패 → REMAP 뒤 재검사 PASS	두 레인 이상 → 링크 bin
4	읽기 레벨링	모든 레인 eye 폭 2 코드 이상	링크 bin
5	ECC 디코더	self-test 78개, 주입대로 정정 · 정정 불가	로직 bin
6	배열 (ECC off)	블록마다 March C- 실패 0	→ 7 수리
7	수리	BIRA → apply → 재시험 PASS → fuse program	수리 불가 → 배열 bin
8	hard repair 확인	리셋 → fuse 로드 → 재시험 PASS	fuse bin
9	mission	ECC on 트래픽, EVCNT · MISR 기대값	FA: ECC_ADDR · trace

- **3이 6보다 먼저**: DQ 핀 stuck-at-1이면 모든 r0이 실패한다(T13). 링크를 먼저 고쳐야 배열 고장과 구분된다
- **6은 ECC off**: 켜면 1 bit 배열 고장이 정정되어 보이지 않는다(T15의 stuck bit)
- **시험 시간**: March C- 연산당 13.2클럭(ffc073d 측정) → 칩 배열 전체 약 2.16M클럭 = 2.7 ms(추정)

March MBIST: 컨트롤러를 거쳐 한 연산씩, 참조 모델과 같은 검출표

hsl_mbist · bank 하나, 최대 16 row × 16 col · MATS+ / March X / March C- · CSR MBIST · MBIST_LOG · MBIST_BITS (commit f018966)

알고리즘 · 8워드 실행 (T13)

알고리즘	요소	연산/워드	8워드
MATS+	{⇳ (w0); ⇑ (r0,w1); ⇓ (r1,w0)}	5	507클럭
March X	{⇳ (w0); ⇑ (r0,w1); ⇓ (r1,w0); ⇳ (r0)}	6	616클럭
March C-	{⇳ (w0); ⇑ (r0,w1); ⇑ (r1,w0); ⇓ (r0,w1); ⇓ (r1,w0); ⇳ (r0)}	10	1,021클럭

연산을 하나씩 내는 이유

- 컨트롤러는 다른 주소의 순서를 바꾼다(FR-FCFS, 쓰기 배치). 여러 개를 띄우면 aggressor 쓰기가 victim 읽기보다 늦게 배열에 닿아 coupling fault를 놓친다
- 쓰기는 응답 → q_empty(WR이 핀으로 나감) → 4클럭, 읽기는 응답을 기다린다. 연산당 약 13.2클럭 (March C- 64워드 8,443클럭, ffc073d)
- 실제 HBM의 배열 MBIST는 core die에서 배열을 직접 몬다. 여기서는 base die → 핀 → 배열 경로 전체를 시험한다

STA와 리뷰가 바꾼 것

- 첫 구현은 주소를 조합으로 만들었다(가변 shift · row 덧셈 · XOR 맵, 0.78 ns). 컨트롤러 req_ready 앞에 붙어 −0.749 ns, 실패 끝점 1,752개 → S_LOAD에서 한 클럭 먼저 계산해 플롭에서 냄
- 레인 BIST와 포트 공유: 다른 엔진이 busy일 때 start를 버리면 읽기 레벨링이 멈춘다 → start를 잡아 두었다가 포트가 비면 냄, 레인 BIST 우선
- 면적 2,849 μm² · FF 264 (실패 로그 16 × 8 bit와 중복 비교기 16개가 큰 몫)

배열 고장 24개 검출 (T13 = tb/march_ref.py 참조)

고장 (victim bit 3, aggressor bit 7)	개수	MATS+	March X	March C-
SA0 · SA1	2	2	2	2
TF ↑ · ↓	2	1	2	2
CFin (전이 × aggressor 앞 · 뒤)	4	3	4	4
CFid (전이 × 강제 값 × 순서)	8	3	4	8
CFst (상태 × victim 값 × 순서)	8	4	5	8
합계 (칩 = 참조)	**24**	**13**	**17**	**24**

- **MATS+는 TF ↓를 놓친다**: 마지막 w0 뒤에 읽기가 없다. March X는 끝의 ⇳ (r0)으로 잡는다
- **CFid · CFst의 두 순서**(aggressor가 victim 앞 · 뒤)는 두 방향 각각에 r0,w1과 r1,w0이 모두 있어야 잡힌다 → March C-

T13 (f018966, 두 시뮬레이터 같은 결과)

- 고장마다 세 알고리즘의 검출 여부가 참조와 같고, 첫 실패가 victim 워드 · 비트
- 실패 워드 로그: SA1 · SA0 · TF ↑ → 처음 실패한 순서로 03 29 3F, 실패 읽기 7
- DQ5 핀 stuck-at-1: r0 192개 실패, 로그 16개가 차고 넘침 비트, 다른 비트 20202020
- 두 엔진 start를 두 순서로: 둘 다 돌고 PASS, 함께 busy인 클럭 0

BIRA · spare · fuse: 실패 로그로 spare를 고르고 fuse에 굽기

spare row 2 + col 2 · repair 표 → 배열 spare · fuse는 SECDED (39,32), 리셋 뒤 자동 로드 (commit 6c29d60)

흐름

```
MBIST(March C-) ─▶ 실패 워드 로그 16개 ─▶ BIRA ─▶ apply
  ─▶ repair 표 ─▶ 배열 spare                    // soft repair
repair 표 ─▶ FUSE program ─▶ fuse 워드 4개 (SECDED 39,32)
칩 리셋 ─▶ fuse 자동 로드 ─▶ 디코드 ─▶ repair 표     // hard repair
```

hsl_bira: 세 단계

단계	하는 일
1. 넘친 로그	기록하지 못한 실패 워드가 있으므로 수리 불가
2. must-repair	변화가 없을 때까지 반복: 실패가 남은 spare col보다 많은 row → spare row, 반대도 같음
3. spare 순서 탐색	남은 spare의 순서(최대 16가지)마다 로그 순서로 덮어 봄. 모두 덮는 첫 순서를 씀

수리 성공률: greedy 대 순서 탐색 (16 × 16 블록, 맵 3,000개씩)

실패 워드	1 + 1	2 + 2 greedy	2 + 2 탐색 = exhaustive	3 + 3 greedy / exh.
3	31.4%	100%	100%	100% / 100%
4	4.8%	100%	100%	100% / 100%
5	0.6%	63.1%	**73.2%**	100% / 100%
6	0.2%	31.7%	**41.3%**	100% / 100%
8	0.0%	4.9%	5.2%	67.9% / 81.4%
10	0.0%	0.2%	0.2%	27.7% / 41.0%

- **첫 구현은 greedy**였다. Python 참조(tb/bira_ref.py)가 해가 있는 맵을 놓치는 것을 보여 교체. 모든 해는 spare 순서 하나에 해당하므로 해가 있으면 찾는다(1 + 1 · 2 + 2 전 맵에서 exhaustive와 assert)

soft · hard repair

- repair 표: spare마다 16 bit {valid, bank, row, col}. apply가 BIRA 결과를 넣고 clear가 비운다. 배열은 표에 있는 row · col 접근을 spare로 보낸다
- fuse program: 항목마다 SECDED 인코딩(0 → 1만). 리셋 뒤 로드: 1 bit는 정정해 세고, 2 bit는 그 항목을 버리고 FUSE[15:12]에 표시
- STA: BIRA 개수를 직렬 증가 체인으로 세면 −0.318 ns → 일치 벡터 + 균형 popcount, 레지스터 뒤 결정(항목당 3클럭). fuse 읽기 → 디코더 −0.256 ns → 먼저 레지스터

T14 (6c29d60, 두 시뮬레이터)

- A 실패 8워드: 표 AC10 AC60 AC05, 재시험 PASS / B hard repair: fuse → 표 비움 → 리셋 → 4항목 복원, 재시험 PASS
- C fuse ECC: 1 bit 정정, 2 bit는 그 항목(col 5)을 버려 3워드가 다시 실패
- D 수리 불가(대각선 5워드, 넘친 로그) / E greedy가 못 고치는 맵을 row +10 · +14, col 9 · 10으로 수리
- F 랜덤 12맵: 판정 · 할당이 TB 참조와 같음, 수리 가능 9개(greedy 8) 모두 PASS
- 면적: BIRA 1,975 · repair 1,152 · SECDED 인코더 122 + 디코더 348 μm²

SECDED ECC: ECC 핀 2개, 오류 주입 · 로그, 디코더 self-test

32 bit 버스트마다 (39,32) · 검사 비트는 데이터 beat 옆 ECC 핀 2 × 4 UI, DRAM이 함께 저장 · 기본 off (commit ffc073d)

```
rtl/phy/hsl_ecc.sv 위치 (hsl_phy_mis 안)
쓰기: 원시 워드 –▶ tx_raw –(반 클럭)–▶ 인코더 –▶ ECC 핀 beat
      DBI –▶ lane shift –▶ tx_word –▶ TX 기어 (데이터 + ECC 48 bit)
읽기: RDQS –▶ Gray FIFO 48 bit –▶ DBI 복원 –▶ 레지스터 단 –▶ 디코더
      RD 추적 FIFO(CA 레지스터) –▶ 첫 오류의 호스트 주소
```

기능 · CSR ECC · ECC_INJ · ECC_CNT · ECC_ADDR

기능	동작
정정 · 검출	1 bit 정정 → ECC_CNT[15:0], 2 bit는 그대로 넘기고 [31:16]에 셈(포화)
첫 오류 로그	호스트 주소 · syndrome · 정정 여부를 clear까지 유지
오류 주입	데이터 · 검사 비트 마스크를 다음 쓰기 하나에 XOR → DRAM에 저장
디코더 self-test	깨끗한 1 + 1 bit 39 + 이웃 2 bit 38 = 78개를 데이터 경로 디코더에

STA가 정한 구조와 대가

- 인코더를 컨트롤러 쓰기 경로에 두면 –1.029 ns → 원시 워드를 레지스터로 받고 TX 기어 E1까지 반 클럭에 인코딩(clk_phy +0.044 ns)
- 디코더 앞 레지스터 단(–0.403), self-test 3단 파이프(–1.125, codeword는 읽기 반환 레지스터로 넣어 mux 없음), RD 추적은 CA 레지스터에서(–0.810)
- 대가: 읽기 지연 +1(열린 row 9 → 10). 컨트롤러가 RD를 4개만 띄워 32 RD가 76 → 97클럭. RD_PIPE를 PHY 왕복에 맞추는 것이 해결책

T15 (ffc073d, 두 시뮬레이터)

항목	결과
self-test	78개 PASS. 정정 플래그를 0에 묶으면 FAIL, 39개 틀림
깨끗한 트래픽	bank 3 row 40의 16워드 쓰기 · 읽기, 오류 0
데이터 1 bit (bit 9)	정정되어 12345678, 첫 오류 주소 1062, syndrome 4E
검사 비트 1 bit	정정, 첫 오류 기록은 그대로
2 bit (bit 0, 1)	정정 불가 1, 데이터는 FACE0003 그대로
배열 bit 17 SA0	ECC on: FFFFFFFF로 정정 / off: FFFDFFFF

한계와 면적

- SECDED라 3 bit 이상은 잘못 정정될 수 있다. DRAM 안의 on-die ECC와는 다른 층이다
- 정정 불가는 CSR로만 알린다. 컨트롤러의 DFI 쪽에 오류 입력이 없어 호스트 응답(rsp_err)에 실리지 않는다
- ECC 핀은 DBI와 레인 리페어 밖이다. enable은 컨트롤러가 idle일 때 바꾼다
- 면적 hsl_ecc 1,960 μm² · FF 153(카운터 32, self-test 파이프, 로그), SECDED 인코더 122 · 디코더 359 μm²
- fault-injection decoder BIST는 공개 자료의 개념으로 다시 짰다

DFD: trace · 카운터 · MISR, 심은 버그를 trace로 찾기

hsl_dfd · CA 핀 레지스터, 읽기 데이터, ECC · PRBS · MBIST 이벤트를 관측 · CSR TRACE · TRACE_DATA · EVCNT0/1 · MISR (commit 796d969)

기능

기능	동작
trace 16 × 32 bit	명령이나 이벤트가 있는 클럭만 기록. {ts 8, 행 CA, 열 cmd · bank, 플래그}. bank 필터
trigger · pre / post	UE · CE · PRBS · MBIST · bank 명령 · 수동. trigger 앞 칸이 다 찬 뒤에만 받고 post개 뒤 정지
명령 카운터	핀의 ACT · RD · WR · REFpb, 16 bit 포화
MISR	읽기 데이터 32 bit signature (x^32 + x^22 + x^2 + x + 1)

디버그 순서 (T16) · 버그를 모른다고 두고 칩 관측 수단만 씀

- **증상**: ECC on 랜덤 읽기 200개. 호스트 데이터는 모두 맞지만(정정) 정정 오류 32개. 첫 오류 주소 1830 = bank 3
- **좁히기**: bank 3만 읽는 트래픽 → 정정 오류가 모두 bank 3 읽기
- **trace**: trigger = ECC 정정, 필터 = bank 3, post 1. 3번 잡음
- **읽기**: 세 번 모두 REFpb b3 → 8클럭 뒤 ACT → 11클럭 뒤 첫 RD → 그 RD 8클럭 뒤 오류. REFpb 전의 RD 7개는 깨끗 → refresh 뒤 첫 읽기 가설
- **확인**: 심은 버그(bug_refdis: REFpb 뒤 16클럭 안 RD의 bit 5 반전)를 끄면 오류 0

T16 trace 덤프 (TRACE_DATA, 세 번 중 하나)

```
t= 33  RD b3  data
t= 35  RD b3  data
t= 37  RD b3
t= 40  RD b3  data       // REFpb 전 RD 7개: 오류 없음
t= 46  RD b3
t= 49  PRE b3  data
t= 52  REFpb b3
t= 60  ACT b3 r60        // +8 = tRFCpb
t= 63  RD b3             // REFpb +11
t= 65  RD b3
t= 67  RD b3
t= 69  RD b3
t= 71  ECC-CE  <- trigger // 첫 RD +8 = 핀 RD → 디코더 플래그
t= 72  RD b3  data
```

면적 · 한계

- 면적 6,019 μm² · FF 701. 거의 trace 512 bit 플롭이다. 실제 칩이라면 trace는 SRAM 매크로
- trace 항목에 column 주소가 없다. 이벤트는 카운터 변화로 보므로 ECC 이벤트가 핀 RD보다 몇 클럭 늦게 찍힌다
- trigger 중 UE · PRBS · MBIST · bank 명령은 시험하지 않았다 (hsl_dfd line 75%)

IEEE 1500 방식 시험 포트, STA가 바꾼 DFT · ECC RTL

hsl_w1500: WIR · WBY · ID · 41 bit CSR WDR, WRCK를 clk로 오버샘플 (commit 14fec6e) · STA: Nangate45 + OpenSTA 3.1

WIR (3 bit, 리셋 BYPASS) · 코드와 배치는 이 설계가 정함

WIR	레지스터	동작
000 BYPASS	WBY 1 bit	한 비트 늦춰 통과 (그 외 코드도)
001 ID	32 bit	capture에서 48534C31 (HSL1)
010 CSR	41 bit {we, addr 8, data 32}	update에서 쓰거나 읽음. 읽기는 scan 두 번

설계 선택과 T17

- **오버샘플**: 입력 7개를 2-FF 동기화하고 WRCK 엣지를 clk에서 찾는다. WRCK ≤ clk / 8 → STA에 새 클럭도 CDC도 없다. 대가는 WRCK 속도 상한
- **CSR 전체 접근**: 한 WDR로 MBIST · BIRA · repair · fuse · ECC · trace · REMAP에 닿는다. 접근하는 1–2 클럭 동안 CSR 버스를 잡는다
- **리셋**: rst_sync_n에 둬서 soft reset에도 WIR 유지. WRSTN이 BYPASS로 되돌린다
- **T17**: ID HSL1, WBY B2 → 64(한 비트 지연), CSR 읽기, March C-를 직렬 포트만으로 시작해 폴링 두 번(82 bit씩)에 PASS
- **없는 것**: WBR · EXTEST. DBI · RED 핀 고장 시험(레인 BIST로 안 보임)이 남았다
- 면적 1,369 μm² · FF 170 (shift · update 레지스터, 읽기 보관, 동기화기)

STA가 짚어 고친 경로 (모두 800 MHz 통과로 바뀜)

경로	slack	조치
MBIST 조합 주소 → 컨트롤러 req_ready	−0.749	한 클럭 먼저 계산, 플롭에서 냄
BIRA 항목 → 개수 직렬 체인 → 결정	−0.318	균형 popcount, 레지스터 뒤 결정
fz_rdata → SECDED 디코더 → repair 표	−0.256	fuse 읽기 데이터 먼저 레지스터
쓰기 데이터 → ECC 인코더 → TX 워드	−1.029	원시 워드 레지스터 + 반 클럭 인코딩
읽기 FIFO → DBI → 디코더 → rddata	−0.403	디코더 앞 레지스터 단
self-test → 디코더 → 실패 카운터	−1.125	세 단계 파이프라인
ccmd → 루프백 판정 → FIFO pop	−0.810	RD 추적 · 루프백을 CA 레지스터에서

칩 STA 결과 (14fec6e)

클럭 · 검사	결과
clk_phy 1.6 GHz	+0.044 ns (최악: ECC 인코더 반 주기)
rdqs 0.625 ns · FIFO 크로싱	+0.106 ns · Gray 포인터 0.104 / 0.625 ns
clk 800 MHz, HSL 로직	모두 통과
clk 800 MHz, 컨트롤러	−0.930 ns, 2.18 ns = 459 MHz
원인 → 방향	선택 + 타이머 갱신 한 클럭 루프 → DFI 1 : 2

11 Summary

핵심 요약 및 향후 과제

Spec → Architecture → RTL → Verification → PD Guide 전 과정 결과와 미해결 항목 정리

핵심 요약

Spec 수치를 문서 → `hsl_pkg` → RTL → TB → SDC까지 단일 값으로 일관 관리하고,
그 정합성을 Simulation · Waveform · 합성 결과로 검증

향후 과제

- 컨트롤러 DFI 1 : 2 (400 MHz, 클럭당 명령 슬롯 2개) — 지금 459 MHz에서 닫힘
- RD_PIPE를 PHY 왕복에 맞춤 — ECC로 늘어난 읽기 지연 회수 (32 RD 97클럭)
- 1500 WBR · EXTEST로 DBI · RED 핀 시험, 로직 스캔 · ATPG
- deskew를 읽기 경로에 연결, 시험하지 않은 DFD trigger 소스 검증
- 컨트롤러 host 경계 레지스터 슬라이스, starvation · RFM 경로 테스트

권용현

010-6408-8646 · kyongh0331@naver.com

github.com/yhkwon-git/High_Speed_Link_Lite